

HTTP & REST Fundamentals

Reference Notes — Behind-the-Scenes Overview | Bookstore Full-Stack Platform (Pre-Stage A)

1.

Q: Give a real-world example of the URL structure (scheme/host/port/path).

Applying it to what we just covered: when you go to `github.com/naveenKumar0408/Learning_with_AI` in your browser —

`https://github.com/naveenKumar0408/Learning_with_AI`

scheme | host | path

Scheme: `https` — GitHub forces encrypted connections, no plain `http` option.

Host: `github.com` — your browser does a DNS lookup, gets back an IP like `140.82.121.4`. GitHub actually runs hundreds of servers behind that one hostname; DNS hands you one via load balancing, you don't control which.

Port: not shown, defaults to `443` since it's `HTTPS`.

Path: `/naveenKumar0408/Learning_with_AI` — this isn't a real folder path on GitHub's disk. GitHub's backend takes that string, looks up which repo it maps to in a database, and returns the rendered page. Proof it's logical, not literal: try a repo that doesn't exist, e.g. `github.com/naveenKumar0408/asdkjaskjd` — you get a `404`, because the server code checked and found nothing, not because a folder was missing.

Real TCP/TLS moment you can actually see: open Chrome DevTools (F12) → Network tab → reload `github.com` → click the first request → Timing tab. It'll show you actual milliseconds spent on DNS lookup, TCP connection, TLS handshake, then the request/response — the exact sequence we just walked through, with real numbers.

2.

Q: Why is HTTP stateless for scalability? What is session memory, what is a session, why would you need to hit the same server, and how does statelessness make it scalable?

Analogy: Imagine a bank teller who remembers your face and your account balance in their own head after you walk up once. Every time you come back to that specific teller's window, they already know who you are — no need to show ID again. But if that teller goes on break and you get a different teller, that new teller has zero memory of you. You'd have to explain everything from scratch, or worse, get turned away because "only teller #3 knows your case."

Session memory (the old/stateful way): Some servers, when you log in, store your info — "this user is logged in, here's their cart, here's their last action" — in that server's own RAM. This is called a session, and it lives on one specific machine. Every future request from you has to land on that exact same machine again, or the new machine has no idea who you are — it never got told.

Why "hit the same server" becomes a requirement: if you have 10 servers behind a load balancer (a traffic router that spreads requests across machines), and your login session is sitting in Server #3's RAM, then every single request you make afterward must be routed specifically back to Server #3. The load balancer has to track "this user → this server" (called sticky sessions) instead of just picking whichever server is least busy. That's extra complexity, and it creates a weak point — if Server #3 crashes, your session data is just gone.

How statelessness fixes this: with HTTP being stateless, no server holds any memory about you. Your JWT (from Stage E) carries everything the server needs to know who you are, right there in the request. That means any server in the cluster (a cluster = a group of servers all capable of running the same app) can handle any request from you — request #1 can hit Server #3, request #2 can hit Server #7, and both work identically, because neither server needed to "remember" anything — you handed them what they needed each time.

That's the scalability win: the load balancer is free to send traffic wherever's fastest, servers can be added/removed/restarted without losing anyone's session, and no single server becomes a point of failure for a specific user.

3.

Q: What does it mean that every ASP.NET Core controller action is one half of a request/response cycle?

Analogy: Think of a controller action like the teller window itself (not the teller's memory — the window/counter where the transaction happens). A request walks up to that window, the action processes it, and a response walks back out. The action is that single exchange — it doesn't exist before the request arrives, and it's done once the response is sent.

Concretely: In ASP.NET Core, a controller action is a method like:

```
[HttpGet("{id}")]
public IActionResult GetBook(int id) // this method = one half
{
    var book = _repository.GetById(id);
    if (book == null)
        return NotFound(); // this return = the other half

    return Ok(book); // response, built here
}
```

The request — method (GET), path (/books/42), any headers/body — is what triggers this method to run, and its data lands in the method's parameters (id here came from the URL path).

The response is whatever the method returns — Ok(book) becomes a 200 OK with the book as JSON body, NotFound() becomes a 404.

So "one half of the cycle" means: the action doesn't handle both sides independently — it's the middle point where an incoming request gets translated into logic (query the DB, run some rule) and that logic's result gets translated back out into an outgoing response. One method call = one complete request → response round trip, nothing carried over from the last call (ties directly back to statelessness — this method has no memory of the previous request either).

4.

Q: What is TLS?

Analogy: Imagine mailing a letter in a clear envelope vs. a sealed, tamper-proof one. Anyone along the postal route could read a clear envelope's contents or swap the letter inside without you knowing. TLS is what turns your HTTP traffic from the clear envelope into the sealed, tamper-proof one.

TLS (Transport Layer Security) is a protocol that sits between TCP and HTTP, and does three things once the connection is open:

1. **Encryption** — scrambles the data so anyone intercepting it (your ISP, someone on the same WiFi, a router in between) sees gibberish, not your actual request/response content.
2. **Authentication** — proves the server is actually who it claims to be, via a certificate (issued by a trusted authority) — so you know github.com is really GitHub's server, not an imposter intercepting your connection.
3. **Integrity** — guarantees the data wasn't tampered with in transit; any modification gets detected.

The handshake: client and server exchange certificates, agree on an encryption algorithm, generate a shared secret key — all before a single byte of your actual HTTP message is sent. This is the step that shows up in DevTools' Timing tab as "SSL/TLS."

Real example: open any site in your browser, click the padlock icon next to the URL → "Connection is secure" / "Certificate is valid." That's TLS actively working — the padlock literally means "TLS handshake succeeded, this connection is encrypted and the server's identity is verified." https = HTTP running over TLS. Plain http has none of this — anyone on the network path can read or alter that traffic.

Your BookstoreApi will use HTTPS by default in ASP.NET Core's dev environment.

5.

Q: Is it true that nothing technically stops a server from modifying a resource using the GET method — it's just a convention that GET should be read-only?

Yes — HTTP itself has no mechanism that enforces GET being read-only.

```
[HttpGet("{id}/delete")]
public IActionResult DeleteBook(int id) // technically works, but wrong
{
    _repository.Delete(id);
    return Ok();
}
```

That code runs fine. GET does not technically prevent side effects — it's purely a convention/contract the developer is expected to honor, called being "safe" (GET/HEAD/OPTIONS are supposed to be side-effect-free) and "idempotent" (calling it once or a hundred times gives the same result, no cumulative change).

Why breaking this convention is genuinely dangerous, not just "bad style": a lot of infrastructure assumes GET is safe and acts on that assumption automatically:

- **Browsers prefetch** — hovering a link, or a browser's "predictive prefetching," can fire a GET request without you ever clicking. If that GET deletes something, it gets silently deleted just from prefetching.
- **Search engine crawlers** — Google's bot GETs every link it finds on a page. A GET-based delete link would let a crawler wipe your data just by indexing it.
- **Caching layers / CDNs / proxies** — many automatically cache GET responses and may even replay them, since they assume GET has no side effects worth re-triggering carefully.
- **Retries** — if a GET times out, clients/browsers commonly auto-retry it assuming it's harmless to repeat. A GET that deletes something would delete it twice, or error out oddly, on retry.

So: not enforced, but it's a load-bearing convention — the entire web's infrastructure (browsers, crawlers, proxies, caches) is built assuming it's honored, so breaking it doesn't just look sloppy, it causes real, hard-to-debug bugs.

What is "server state"? Any data on the server that persists and can change — the actual condition of the system at a given moment. For the Bookstore project: rows in the database (a Product's price, a Book's stock count, whether an Order exists), session/cache data if any is held server-side, files on disk the server manages, any in-memory counters or flags the app tracks across requests. If someone wrote GET /products/7/increment-stock, every GET call increases stock by 1 — that's modifying state, and now it's no longer safe, because a browser prefetch or crawler hitting that link 5 times just changed inventory 5 times without anyone intending it.

6.

Q: Behind-the-scenes: what is a web application, what is a server machine, and what is the DB's role?

Web application — correct that it's an application accessible over the internet. An application whose logic runs on a remote machine (the server), accessed over a network (the internet) via a client, rather than running entirely on your own device (contrast: a desktop app like Notepad, which needs no server at all).

Server machine — just a computer, no different in principle from your laptop. The real distinctions:

- **No UI/monitor typically attached** — most servers are "headless," managed remotely via SSH/RDP, not sat in front of with a keyboard.
- **Runs continuously** — unlike your laptop, it doesn't sleep or shut down; it's built to stay up 24/7.
- **Runs server software**, not just "the application" generically. A server machine typically runs: an OS (Linux/Windows Server); a web server / app host (Kestrel, for ASP.NET Core — this is the actual process listening on a port for HTTP requests); your application code running inside/behind that host.

So "the server" isn't one blob — it's a machine, running an OS, running a hosting process, running your

compiled app. Each layer has a job.

DB — the DB is usually a separate server/machine (or at least a separate process) from the application server. In Stage B/C, SQL Server and BookstoreApi aren't the same thing running in the same process — the API server makes a network connection to the database server (even if both happen to sit on the same physical machine during local dev, they're logically and often physically separate). That separation is why connection strings exist (Server=...;Database=...;) — the API is a client to the database, the same way a browser is a client to the API. Client/server is a recursive pattern, not a one-time relationship.

This is directly relevant to the full picture (components, sessions, clusters): BookstoreApi, when deployed, is really: Kestrel process → app code → separate DB server connection — and in a cluster, multiple copies of that Kestrel+app combo all talk to the same shared DB server, which is part of why statelessness matters — the DB is the one thing all instances share, nothing else is.

7.

Q: Is signing up (creating credentials) about generating identity, and signing in (using credentials on return visits) about using that identity — i.e. is this all part of authentication/authorization, and does it apply beyond just admin vs. regular-user roles?

Sign up = creating an identity. The server stores your credentials (username/email + a hashed password — never plain text, important detail for Stage E) in the database. This is the one-time act of the system learning "this identity exists."

Sign in / login = authentication happening. You present credentials, the server checks them against what's stored, and if they match, it establishes "yes, this request really is coming from that identity" — and issues you a token (JWT) as proof, so you don't have to resend username/password on every future request (that's what the token replaces).

Every visit after that = you present the token, server authenticates you again (per request, since it's stateless — no memory of the last time), then authorizes whatever specific action you're trying to do.

Authentication/authorization isn't a concept exclusive to "admin vs regular user" — that was just one example of authorization (role-based). The bigger picture:

- **Authentication** = the whole identity lifecycle: sign up (create identity) → sign in (prove identity) → every subsequent request (re-prove identity via token)
- **Authorization** = a layer on top, decides what an already-authenticated identity can do — which can be role-based (Admin vs Customer), ownership-based (you can edit your own order but not someone else's), or any other rule the app defines

Even a site with no roles at all (everyone's just "a logged-in user," no Admin/Customer split) still has authentication — sign up, sign in, token on every request. Authorization only becomes visible as a separate concept once different identities are allowed to do different things. In Bookstore, Stage E gives you both halves: Identity handles sign-up/sign-in (authentication), and [Authorize(Roles = "Admin")] handles the permission layer (authorization) on top of it.

8.

Q: Without signing up, a user can't see the full features made accessible to a signed-up user — does this also deal with authorization?

Yes — that's authorization too, just a different axis of it than the role-based example. This is the distinction between an anonymous/unauthenticated user and an authenticated user:

- No sign-up/login at all → not authenticated → the app authorizes you for only the public subset (browse books, view listings)
- Signed up and logged in → authenticated → now authorized for the "logged-in user" tier (add to cart, place orders, view your order history)
- Signed up, logged in, and Admin role → authorized for an even wider tier (manage books, view all orders)

So authorization isn't just "Admin vs Customer" — it's really tiers of access, and "authenticated vs anonymous" is itself the very first, broadest tier. Being logged in at all is already a permission level compared to being anonymous — you don't need roles to have authorization; you just need at least two different levels of "what am I allowed to see/do," and anonymous-vs-logged-in already qualifies as exactly that.

In Bookstore (Stage E/F), this maps directly to real endpoints:

- GET /books → [AllowAnonymous] — public browsing, no auth needed
- POST /orders → requires a valid JWT — must be logged in, any role
- DELETE /books/7 → requires JWT and Admin role — the narrowest tier

Each layer is authorization, progressively narrowing who's allowed in.

9.

Q: What exactly is an endpoint, and is it different from or the same as a URL?

Related, not identical — an endpoint is a broader concept than a URL is just one piece of.

Analogy: Think of a URL as a street address, and an endpoint as "that address + a specific service offered there." 123 Main St is an address, but "123 Main St, the pizza counter" and "123 Main St, the dry cleaning counter" (same building, different windows) are two different endpoints if they're two separate services — even though the address is the same.

Precise definition: an endpoint = a specific combination of URL path + HTTP method that the server exposes as one distinct operation. Not the URL alone.

GET /books/42 → one endpoint (fetch book 42)

PUT /books/42 → a different endpoint (update book 42)

DELETE /books/42 → yet another endpoint (delete book 42)

All three share the exact same URL path, but they're three separate endpoints, because an endpoint is

defined by the method+path pair together, not the path in isolation.

In ASP.NET Core terms (Stage C): every [HttpGet], [HttpPost], [HttpPut], [HttpDelete] method in a controller is one endpoint. BookstoreApi will end up with dozens of endpoints, many sharing the same base path (/books/{id}) but differing by method.

10.

Q: REST doesn't require JSON, any specific framework, or authentication of any kind — what does this mean?

This means REST is an architectural style, not a technology — it doesn't mandate any specific implementation detail.

Doesn't require JSON: REST just says "resources have representations" — JSON is simply the most popular format for that representation today. A REST API could equally return XML, plain text, HTML, even images — as long as it follows the resource/verb/stateless conventions, it's still REST. Older APIs (2000s-era) were commonly XML-based and still called RESTful. BookstoreApi will use JSON purely because it's the modern default (and what Angular's HttpClient expects easily) — not because REST forces it.

Doesn't require any specific framework: REST is a set of rules about how to design an API's shape — it doesn't care whether you implement it in ASP.NET Core, Node/Express, Spring Boot, Django, or raw sockets you hand-code yourself. Any language/framework capable of speaking HTTP can implement REST. ASP.NET Core is just the chosen tool for building one.

Doesn't require authentication of any kind: REST is silent on security entirely. A REST API can be completely open, no login required at all (e.g. a public weather API — anyone hits GET /weather/london, no token needed). Authentication (JWT, API keys, OAuth, whatever) is a separate concern layered on top when the application needs it — not something REST itself demands. Bookstore needs auth because of the choice to protect admin actions (Stage E) — REST as a style has no opinion on that.

The core point: REST constrains the shape and behavior of the API (nouns/verbs, statelessness, uniform interface, etc.) — everything else (data format, language, security model) is an independent decision the developer makes on top of REST, not part of REST itself.

11.

Q: In places API is referred to as the Server — but API is not the server, right? The server is what listens for requests, processes them, and returns a response?

Right — API and server are not the same thing, though they get used loosely/interchangeably in casual conversation. Precisely:

Server = the machine/process that listens on a port, accepts connections, and handles requests. It's the infrastructure — Kestrel (ASP.NET Core's built-in web server) running the compiled app, listening on port

5000/5001/443.

API = the actual contract/interface — the set of endpoints, their expected inputs, and their outputs, that the application exposes. It's not a running process; it's the definition of what operations are available (GET /books, POST /orders, etc.) and how to use them.

How they relate: the server is what physically runs and executes the API. The API is the thing being served. The same API contract could be exposed by different server setups (dev server vs production server vs a different host entirely), and the API itself (the endpoints, their shape) doesn't change — only the infrastructure hosting it does.

Why people say "API" loosely to mean "server": because in everyday conversation, "call the API" and "call the server" end up meaning the same practical action (send a request to that address), so the terms blur. But structurally:

- **Server** = the running process, the infrastructure, the "listener"
- **API** = the interface/contract that server exposes — the actual set of endpoints and rules for using them
- **Application code** = the controllers/business logic that the API contract maps to, which the server hosts and executes

So BookstoreApi has: a server (Kestrel) → hosting the app → exposing an API (the controllers' endpoints). Three distinct layers, commonly flattened into one word in casual speech, but worth keeping separate — this becomes very concrete in Stage C when Kestrel/hosting is configured separately from writing controllers.

12.

Q: [Revisit item, expanded on request] What are Cookies?

Analogy: A cookie is like a stamp on your hand at a concert venue. You show ID once at the entrance (login), they stamp your hand, and for the rest of the night, bouncers just check the stamp — they don't ask for ID again. The stamp is small, sits on you, and gets checked automatically wherever you go inside the venue.

Technical: A cookie is a small piece of data the server tells the browser to store (via a Set-Cookie response header), and the browser automatically re-sends on every subsequent request to that same domain — no extra code needed on the client side, it's built into how browsers work.

```
Response: Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
```

After this, every request the browser makes to that domain automatically includes:

```
Cookie: sessionId=abc123
```

Cookies vs JWT (the comparison that actually matters): Cookies are a transport mechanism (a way to carry a value automatically); JWT is a data format (a self-contained, signed token). They're often confused because they're commonly used together — historically, session-based auth stored a session ID in a cookie, and the server looked up that ID in server-side memory (session state — ties back to

statelessness). JWT-based auth (Stage E) usually skips cookies entirely and puts the JWT in the Authorization header instead, specifically because that avoids server-side session memory and keeps things stateless.

Where cookies still matter even in a JWT-based app: refresh tokens are sometimes stored in an HttpOnly cookie (JavaScript can't read it, protecting against XSS attacks) while the short-lived access JWT is kept in memory and sent via the header. Not something to implement yet — just good to recognize the pattern when seen in Stage E material.

13.

Q: [Queued deep-dive, expanded on request] What is a Server (components), what is a Session, what is a Cluster?

Server components: A "server" you interact with is really a stack of layers, not one thing:

- **Physical/virtual machine** — the actual hardware (or a slice of hardware, if it's a VM in the cloud)
- **Operating system** — Linux or Windows Server, manages hardware resources
- **Runtime** — .NET runtime, installed on that OS, capable of executing the compiled app
- **Web server/host** — Kestrel, a lightweight process that binds to a port, speaks raw HTTP, and hands parsed requests to the application code
- **Application** — the controllers, services, EF Core context — the actual business logic
- **Database server** — usually a separate machine/process (SQL Server), which the app connects to as a client

Session: When an app is "stateful" (the older model), the server keeps data about a specific user in its own RAM, tied to a sessionId (often delivered via cookie). E.g., a shopping cart stored server-side: `Session["cart"] = [...]`. Problem: that RAM lives on one specific machine. If you have multiple servers, only the one machine that created your session actually has your cart — hence "sticky sessions" being required, which is the scalability problem statelessness solves.

Cluster: A cluster is a group of servers (multiple machines, each running an identical copy of the app) sitting behind a load balancer — a router that decides which server handles each incoming request. Purpose: handle more traffic than one machine could alone, and survive a machine failing (if Server #3 crashes, #1/#2/#4 keep serving traffic, nobody notices). This only works cleanly if the app is stateless — any server can pick up any request — which is precisely why REST's statelessness constraint isn't academic, it's what makes clustering possible at all. BookstoreApi, if deployed to a cloud environment (Azure, AWS), would typically run as multiple instances in exactly this kind of cluster, all pointed at the same single SQL Server database — the DB being the one shared source of truth, everything else disposable/replaceable.

14.

Q: [Must-visit concept, expanded on request] What is the chain of middleware in ASP.NET Core?

Analogy: Think of an airport security line: check-in counter → ID check → baggage scan → boarding gate. Each checkpoint does one job, decides whether you continue to the next, and can stop you entirely

(denied boarding) before you ever reach the gate.

Technical: Middleware is a pipeline of components that every incoming HTTP request passes through, in order, before it ever reaches the controller action — and the response passes back through them in reverse order on the way out. Each middleware can: inspect/modify the request; short-circuit and return a response early (never reaching the controller); pass control to the next middleware in line; do something after the next middleware returns (logging response time, for example).

Typical order in an ASP.NET Core app (Stage C/E, in Program.cs):

```
app.UseHttpsRedirection(); // force HTTPS
app.UseSerilogRequestLogging(); // log every request (Stage C)
app.UseAuthentication(); // WHO are you? (checks JWT, sets User identity)
app.UseAuthorization(); // WHAT are you allowed to do? (checks [Authorize])
app.MapControllers(); // finally reaches your controller actions
```

Why order matters, concretely: UseAuthentication must run before UseAuthorization — you can't check permissions for an identity you haven't established yet. If a request fails authentication, the pipeline can stop right there and return 401, never even reaching the controller code — directly explaining how 401/403 actually get produced mechanically, not just conceptually.

15.

Q: [Must-visit concept, expanded on request] Consolidated: Authentication and Authorization.

This ties together everything covered piecemeal across the conversation:

- **Authentication** = establishing identity. Sign-up creates it, sign-in proves it, JWT (sent in the Authorization header on every request, because HTTP is stateless) re-proves it each time.
- **Authorization** = deciding what an established identity can do. Ranges from the broadest tier (anonymous vs logged-in) to narrow role-based checks (Admin vs Customer) to ownership checks (can only edit your own order).
- **401 vs 403** = authentication failure vs authorization failure — who-are-you vs what-are-you-allowed. 401 Unauthorized = authentication failure — token missing, expired, malformed, or invalid signature; the server can't establish identity at all. 403 Forbidden = authorization failure — token is valid, server knows exactly who you are, but the role/permissions don't cover this specific action.
- **Mechanically**, both are enforced via middleware (UseAuthentication / UseAuthorization) sitting in the pipeline before the controller ever runs, plus [Authorize] attributes on individual actions specifying the required role/policy.

All of this gets built concretely in Stage E — ASP.NET Core Identity (sign-up/sign-in, password hashing), JWT issuance on login, and [Authorize(Roles = "Admin")] on the book-management endpoints.

Reminder — Revisit at Stage E (Auth/JWT)

Q: Is the Authorization: Bearer eyJ... header part of the request header?

Authorization: Bearer eyJ... is a header, same category as Host. It carries the client's auth token (a JWT, given the eyJ prefix — that's base64 for {" , the start of JSON) so the server can identify/authorize the caller before processing the request. This gets built directly in Stage E when JWT auth is wired up.